Lab 1 — NTFS File Server Lab

Active Directory · NTFS Permissions · SMB File Services · Group Policy · Terraform · PowerShell

Field	Value
Domain	lab.local	Region: Central US
VMs	DC01 (Domain Controller) · FS01 (File Server) · CLIENT01 (Windows 11 Workstation)
Deploy Time	10–15 min (terraform) + 15–20 min (configure-lab.ps1)
Cost	~$0.15–$0.25/hr while all three VMs are running
Relationship to other labs	Lab 2 (RBAC) builds directly on top of this lab's infrastructure

This SOP is self-contained. Every file you need is printed in full below. No repo required. No downloads beyond Terraform and the Azure CLI.

How This Lab Fits Into the Series

There are three labs in this series. Understanding how they relate before you start prevents confusion about why certain choices are made — particularly around remote state, resource group names, and VM names that are referenced by later labs.

Lab	What it deploys	Relationship
Lab 1 — NTFS File Server	DC01, FS01, CLIENT01, VNet, NSG, Key Vault in RG-FileServerLab	Standalone — creates all infrastructure from scratch
Lab 2 — Azure RBAC	3 role assignments on FS01 only — no new VMs	Depends on Lab 1 — reads Lab 1 resources using data sources, reuses Lab 1 storage account
AUM Lab — Azure Update Manager	DC01, WS01, WS02, VNet, Key Vault in rg-aumlab	Standalone — fully independent from Lab 1 and Lab 2

If you plan to do Lab 2 after this: Stop the VMs instead of destroying when you finish. Lab 2 needs FS01 to already exist. Stopped VMs have no compute charges.

What This Lab Covers and Why It Matters

The Business Problem This Lab Solves

Every organization running Windows infrastructure faces the same challenge: controlling who can access what data. Finance data should only be readable by Finance staff. HR files should be invisible to Sales. IT staff need administrative access everywhere to do their jobs.

The solution — still widely deployed in enterprise environments today, including hybrid cloud environments — is a Windows File Server backed by Active Directory groups with NTFS permissions. This is not outdated technology. It is the backbone of file access control in thousands of organizations.

This lab puts you through the complete workflow a systems administrator follows when building this from scratch using modern cloud infrastructure and Infrastructure as Code.

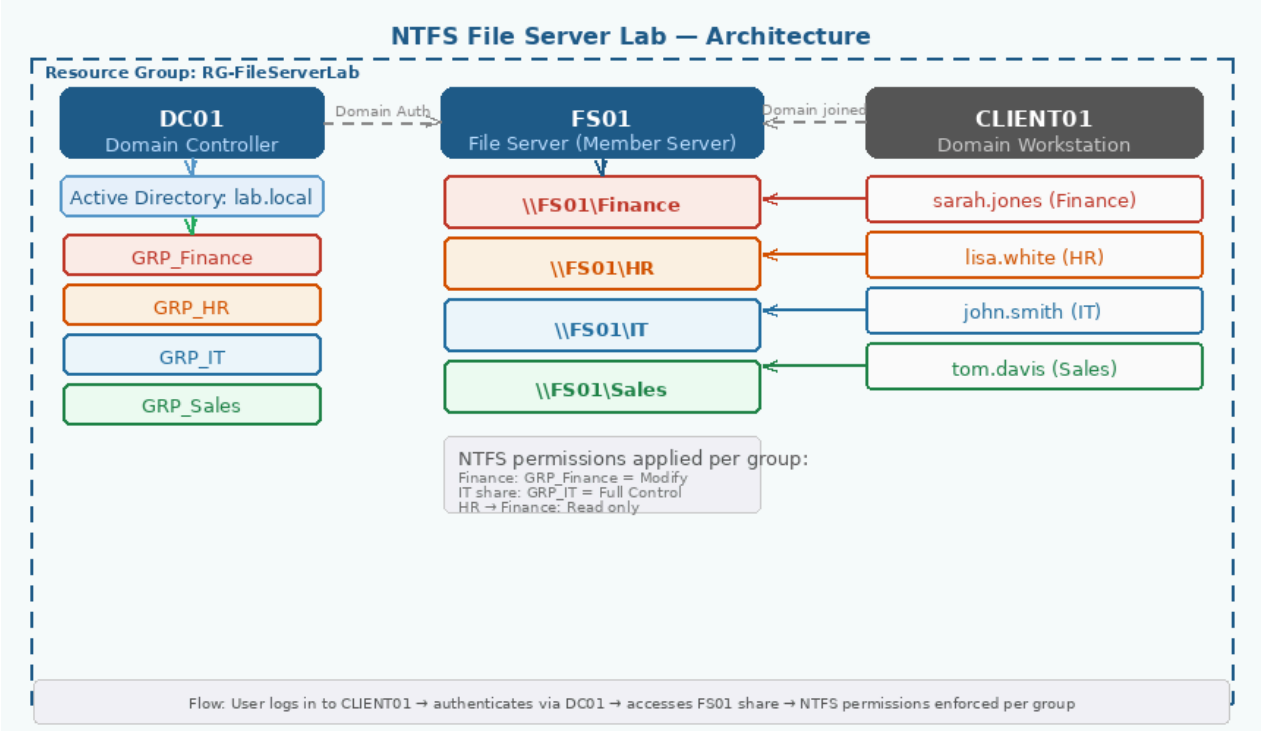
What You Will Learn

Skill	Why it matters in a real environment
Deploy Active Directory with Terraform	In production, AD is provisioned as code so it can be recreated identically across environments without manual configuration drift
Create OUs and security groups	OUs let you apply Group Policy to specific sets of users or computers. Groups let you manage permissions for hundreds of users by changing one group membership instead of editing every file and folder individually
Configure NTFS permissions	NTFS is the actual enforcement layer on Windows file systems. Understanding how inheritance, group permissions, and icacls work is required knowledge for any Windows sysadmin role
Create and secure SMB shares	SMB is the protocol Windows uses for file sharing across a network. Every shared drive in a corporate environment uses SMB. Knowing how share-level and NTFS-level permissions interact is a job interview topic
Deploy infrastructure with Terraform	IaC means the environment is reproducible, version-controlled, and auditable. A sysadmin who can write Terraform is significantly more valuable than one who only uses the portal
Store secrets in Azure Key Vault	Hard-coding passwords in scripts is a security failure. Key Vault is the correct pattern — credentials are stored once, retrieved at runtime, and never written to disk or source control
Use az vm run-command for automation	In real Azure environments you often cannot RDP directly into VMs due to network restrictions. The Azure agent provides a secure, firewall-bypassing channel for executing scripts — this is how production automation works

The Business Scenario Behind the Test Users

The five test users are not arbitrary. They represent a real org chart scenario. Sarah and Mike are Finance staff — they need read/write on Finance data. Lisa is HR — she needs read/write on HR data and read-only on Finance for cross-department reporting. John is IT — he needs full control everywhere to do his job. Tom is Sales — he has no access to Finance or HR data because he doesn't need it. When you test each user's access in Step 9, you are validating that the permission model matches the business requirements.

Architecture



DC01 runs Active Directory, DNS, and Group Policy. FS01 hosts the four SMB shares with NTFS permissions enforced per security group. CLIENT01 is the Windows 11 workstation where test users log in to exercise the full permission chain. All three VMs are on the same subnet inside a single Azure VNet, protected by an NSG that only allows RDP from your specific IP.

Why Each Component Exists

Component	What it does	Why it's needed
Resource Group (RG-FileServerLab)	Container for all lab resources	One resource group means one az group delete cleans up everything. Lab 2 references this group by name — it must stay consistent
VNet (10.0.0.0/16)	Private network for all three VMs	VMs on the same VNet communicate using private IPs without going through the internet. DC01 talks to FS01 and CLIENT01 privately at no cost
Subnet (10.0.1.0/24)	Address range carved from the VNet	All three VMs are on this subnet so they can reach each other directly. The /24 gives 251 usable addresses — more than enough
NSG — Allow RDP from your IP only	Firewall rules at the NIC level	Restricts inbound port 3389 to your IP only. All other inbound traffic is implicitly denied. Prevents the VMs from being reachable from the entire internet
Static IP on DC01 (10.0.1.4)	DC01 always has the same private IP	FS01 and CLIENT01 must point DNS at DC01 to resolve lab.local. A static IP ensures that address never changes after a VM restart or reallocation
Public IPs on all three VMs	External addresses for RDP access from your machine	Without these, VMs are only reachable from within Azure. Public IPs let you connect from your local machine for verification
Azure Key Vault with RBAC model	Encrypted secret storage	Stores the VM admin password so it never appears in a file, CLI argument, or terminal history. configure-lab.ps1 retrieves it at runtime using your logged-in CLI identity
enable_rbac_authorization = true on Key Vault	Activates the modern RBAC permission model	Without this flag, role assignments on the vault are silently ignored by Azure and every secret read or write returns 403 — even if the role assignment appears to exist in the portal
random_id suffix on Key Vault name	Ensures global uniqueness	Key Vault names must be unique across all of Azure — not just your subscription. A random 8-character hex suffix makes collision essentially impossible
time_sleep resources in main.tf	Deliberate pauses after network creation	Azure's control plane can have replication delays after a VNet or NSG is created. Without these pauses, subsequent resources sometimes fail with transient 'not found' errors
CustomScriptExtension on CLIENT01	Enables RDP at the OS level immediately after boot	Windows 11 ships with RDP disabled. The extension runs a one-line PowerShell command that turns it on — without this, RDP attempts time out with no error message

Component	What it does	Why it's needed
az vm run-command in configure-lab.ps1	Executes scripts on VMs without needing RDP or WinRM	The NSG only allows port 3389. WinRM (port 5985) is blocked. az vm run-command sends scripts through the Azure VM agent which is already installed on all Azure VMs and bypasses NSG rules entirely

---

Prerequisites — Verify Before Starting

```
terraform -version   # Must be >= 1.5.0
az version           # Azure CLI — any recent version
powershell -version  # 5.1+ built into Windows

# Confirm you are on the correct Azure subscription
az account show
# If wrong: az account set --subscription "<name or ID>"
```

Step 1 — Create Your Project Folder

Create a dedicated folder. All Terraform files go in the root. PowerShell scripts go in scripts/. configure-lab.ps1 references scripts/ by relative path so this structure must be maintained.

```
New-Item -ItemType Directory -Path "$HOME\ntfs-lab-terraform"
cd "$HOME\ntfs-lab-terraform"
New-Item -ItemType Directory -Path scripts
```

```
ntfs-lab-terraform/
├── backend.tf                    ← remote state — points to Azure Blob Storage
├── versions.tf                   ← provider versions Terraform downloads
├── variables.tf                  ← all input variables including admin_password
├── main.tf                       ← VMs, VNet, NSG, NICs, public IPs
├── keyvault.tf                   ← Key Vault + secret + RBAC assignment
├── outputs.tf                    ← IPs and Key Vault name printed after apply
├── terraform.tfvars.example      ← safe template, safe to commit
├── terraform.tfvars              ← your real values, never commit this
├── .gitignore                    ← prevents sensitive files from being committed
├── configure-lab.ps1             ← the only script you run manually
└── scripts/
    ├── 00-promote-dc.ps1         ← DC01: installs AD DS, promotes to domain
controller
    ├── 01-create-ad-users-groups.ps1  ← DC01: creates OUs, groups, test users
    ├── 02-configure-shares-and-permissions.ps1  ← FS01: SMB shares + NTFS ACLs
    ├── 03-configure-rdp-gpo.ps1 ← DC01: creates RDP Group Policy Object
    ├── 04-domain-join.ps1        ← FS01 and CLIENT01: joins both to lab.local
    ├── 05-verify-ad.ps1          ← DC01: automated PASS/FAIL check of AD objects
    ├── 05-verify-shares.ps1      ← FS01: automated PASS/FAIL check of permissions
```

```
    └── 06-add-rdp-users.ps1     ← CLIENT01: adds domain users to RDP group
```

Step 2 — Create the Terraform Files

📄 **backend.tf**

What is remote state and why use it? Terraform tracks every resource it creates in a state file. Without remote state it sits on your local disk — if you lose it, Terraform loses track of every resource and teardown becomes very difficult. Storing it in Azure Blob Storage means it is encrypted, backed up, and accessible from any machine. Lab 2 (RBAC) uses the same storage account but a different state key — so both labs are independent while sharing the infrastructure cost.

Complete Step 3 first. Create the storage account in Step 3, then come back and replace REPLACE_WITH_YOUR_STORAGE_ACCOUNT_NAME. If you run terraform init before doing this the command will fail with a storage account not found error.

```
terraform {
  backend "azurerm" {
    resource_group_name  = "RG-TerraformState"
    storage_account_name = "REPLACE_WITH_YOUR_STORAGE_ACCOUNT_NAME"
    container_name       = "tfstate"
    key                  = "ntfs-lab.terraform.tfstate"
    # Lab 2 uses key = "rbac-lab.terraform.tfstate"
    # Both labs share the same container without overwriting each other
  }
}
```

📄 **versions.tf**

Provider	Why this lab needs it
azurerm	The Azure provider — creates every Azure resource: VMs, VNet, Key Vault, NSG, etc.
random	Generates a random 8-character hex suffix for the Key Vault name so it is globally unique across all of Azure
time	Provides time_sleep resources used to pause after VNet and NSG creation — prevents transient Azure consistency errors

```
terraform {
```

```
  required_version = ">= 1.5.0"
  required_providers {
    azurerm = { source = "hashicorp/azurerm" version = "~> 3.0" }
    random  = { source = "hashicorp/random"  version = "~> 3.6" }
    time    = { source = "hashicorp/time"    version = "~> 0.11" }
  }
}
provider "azurerm" { features {} }
```

variables.tf

admin_password has no default and is marked sensitive = true. This means Terraform will never print it in plan or apply output. It must be provided as an environment variable — never in a file.

```
variable "location"            { type=string default="Central US"
description="Azure region." }
variable "resource_group_name" { type=string default="RG-FileServerLab"
description="Must stay consistent — Lab 2 references this name." }
variable "vnet_name"           { type=string default="VNET-FileServerLab" }
variable "subnet_name"         { type=string default="Subnet-Servers" }
variable "vnet_cidr"           { type=string default="10.0.0.0/16" }
variable "subnet_cidr"         { type=string default="10.0.1.0/24" }
variable "nsg_name"            { type=string default="NSG-RDP" }
variable "rdp_source"          { type=string default="*"
  description="Your public IP in CIDR format — e.g. 1.2.3.4/32. Find at
whatismyip.com" }
variable "admin_username"      { type=string default="azureadmin" }
variable "admin_password"      { type=string sensitive=true
  description="Set as TF_VAR_admin_password env var — never in a file." }
variable "server_vm_size"      { type=string default="Standard_B2as_v2" }
variable "client_vm_size"      { type=string default="Standard_B2as_v2" }
```

main.tf

Creates all networking and VMs. Every time_sleep resource exists because Azure sometimes has replication delays after network resources are created — resources built immediately after can fail with transient errors. The 45-second pauses prevent this.

```
# Resource Group
resource "azurerm_resource_group" "rg" {
  name     = var.resource_group_name
  location = var.location
}

# Virtual Network — 10.0.0.0/16 provides 65,536 addresses
resource "azurerm_virtual_network" "vnet" {
  name                = var.vnet_name
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name
  address_space       = [var.vnet_cidr]
}
resource "time_sleep" "wait_after_vnet" { create_duration="45s"
depends_on=[azurerm_virtual_network.vnet] }
```

```
# Subnet — 10.0.1.0/24 provides 251 usable addresses
resource "azurerm_subnet" "subnet" {
  name                 = var.subnet_name
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = [var.subnet_cidr]
  depends_on           = [time_sleep.wait_after_vnet]
}

# NSG — only your IP can reach port 3389. All other inbound is denied by default.
resource "azurerm_network_security_group" "nsg" {
  name                = var.nsg_name
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name
  security_rule {
    name="Allow-RDP-3389" priority=100 direction="Inbound" access="Allow"
protocol="Tcp"
    source_address_prefix=var.rdp_source source_port_range="*"
    destination_port_range="3389" destination_address_prefix="*"
  }
  depends_on=[time_sleep.wait_after_vnet]
}
resource "time_sleep" "wait_after_nsg" { create_duration="45s"
depends_on=[azurerm_network_security_group.nsg] }

# Public IPs — Standard SKU required for static allocation
resource "azurerm_public_ip" "dc01"     { name="dc01-pip"     location=var.location
resource_group_name=azurerm_resource_group.rg.name allocation_method="Static"
sku="Standard" depends_on=[time_sleep.wait_after_nsg] }
resource "azurerm_public_ip" "fs01"     { name="fs01-pip"     location=var.location
resource_group_name=azurerm_resource_group.rg.name allocation_method="Static"
sku="Standard" depends_on=[time_sleep.wait_after_nsg] }
resource "azurerm_public_ip" "client01" { name="client01-pip" location=var.location
resource_group_name=azurerm_resource_group.rg.name allocation_method="Static"
sku="Standard" depends_on=[time_sleep.wait_after_nsg] }

# DC01 NIC — STATIC IP 10.0.1.4 so FS01/CLIENT01 DNS never breaks after restarts
resource "azurerm_network_interface" "dc01" {
  name="dc01-nic" location=var.location
resource_group_name=azurerm_resource_group.rg.name
  ip_configuration { name="internal" subnet_id=azurerm_subnet.subnet.id
    private_ip_address_allocation="Static" private_ip_address="10.0.1.4"
    public_ip_address_id=azurerm_public_ip.dc01.id }
  depends_on=[time_sleep.wait_after_nsg]
}
resource "azurerm_network_interface" "fs01" {
  name="fs01-nic" location=var.location
resource_group_name=azurerm_resource_group.rg.name
  ip_configuration { name="internal" subnet_id=azurerm_subnet.subnet.id
    private_ip_address_allocation="Dynamic"
public_ip_address_id=azurerm_public_ip.fs01.id }
  depends_on=[time_sleep.wait_after_nsg]
}
resource "azurerm_network_interface" "client01" {
  name="client01-nic" location=var.location
resource_group_name=azurerm_resource_group.rg.name
  ip_configuration { name="internal" subnet_id=azurerm_subnet.subnet.id
    private_ip_address_allocation="Dynamic"
public_ip_address_id=azurerm_public_ip.client01.id }
  depends_on=[time_sleep.wait_after_nsg]
}

# Attach NSG to each NIC — without this the NSG exists but applies to nothing
resource "azurerm_network_interface_security_group_association" "dc01"     {
network_interface_id=azurerm_network_interface.dc01.id
network_security_group_id=azurerm_network_security_group.nsg.id
depends_on=[time_sleep.wait_after_nsg] }
```

```
resource "azurerm_network_interface_security_group_association" "fs01"     {
network_interface_id=azurerm_network_interface.fs01.id
network_security_group_id=azurerm_network_security_group.nsg.id
depends_on=[time_sleep.wait_after_nsg] }
resource "azurerm_network_interface_security_group_association" "client01" {
network_interface_id=azurerm_network_interface.client01.id
network_security_group_id=azurerm_network_security_group.nsg.id
depends_on=[time_sleep.wait_after_nsg] }

# DC01 — Windows Server 2022 (Azure Edition includes VM agent pre-installed)
resource "azurerm_windows_virtual_machine" "dc01" {
  name="DC01" location=var.location
resource_group_name=azurerm_resource_group.rg.name
  size=var.server_vm_size admin_username=var.admin_username
admin_password=var.admin_password
  network_interface_ids=[azurerm_network_interface.dc01.id]
  os_disk { caching="ReadWrite" storage_account_type="Standard_LRS" }
  source_image_reference { publisher="MicrosoftWindowsServer" offer="WindowsServer"
    sku="2022-datacenter-azure-edition" version="latest" }
  depends_on=[time_sleep.wait_after_nsg]
}

# FS01 — Windows Server 2022
resource "azurerm_windows_virtual_machine" "fs01" {
  name="FS01" location=var.location
resource_group_name=azurerm_resource_group.rg.name
  size=var.server_vm_size admin_username=var.admin_username
admin_password=var.admin_password
  network_interface_ids=[azurerm_network_interface.fs01.id]
  os_disk { caching="ReadWrite" storage_account_type="Standard_LRS" }
  source_image_reference { publisher="MicrosoftWindowsServer" offer="WindowsServer"
    sku="2022-datacenter-azure-edition" version="latest" }
  depends_on=[time_sleep.wait_after_nsg]
}

# CLIENT01 — Windows 11 Pro (ships with RDP disabled — fixed by extension below)
resource "azurerm_windows_virtual_machine" "client01" {
  name="CLIENT01" location=var.location
resource_group_name=azurerm_resource_group.rg.name
  size=var.client_vm_size admin_username=var.admin_username
admin_password=var.admin_password
  network_interface_ids=[azurerm_network_interface.client01.id]
  os_disk { caching="ReadWrite" storage_account_type="Standard_LRS" }
  source_image_reference { publisher="MicrosoftWindowsDesktop" offer="windows-11"
    sku="win11-23h2-pro" version="latest" }
  depends_on=[time_sleep.wait_after_nsg]
}

# Enable RDP on CLIENT01 — Windows 11 has it disabled by default.
# Sets fDenyTSConnections=0 and opens the firewall rule.
# Without this, RDP attempts to CLIENT01 time out with no error.
resource "azurerm_virtual_machine_extension" "client01_enable_rdp" {
  name="enable-rdp" virtual_machine_id=azurerm_windows_virtual_machine.client01.id
  publisher="Microsoft.Compute" type="CustomScriptExtension"
type_handler_version="1.10"
  settings=jsonencode({commandToExecute="powershell -Command \"Set-ItemProperty
-Path 'HKLM:\\System\\CurrentControlSet\\Control\\Terminal Server' -Name
'fDenyTSConnections' -Value 0; Enable-NetFirewallRule -DisplayGroup 'Remote
Desktop'\""})
  depends_on=[azurerm_windows_virtual_machine.client01]
}
```

keyvault.tf

Creates the Key Vault and stores the admin password as a secret. The configure-lab.ps1 script retrieves this at runtime — you never type the password again after terraform apply completes.

```
data "azurerm_client_config" "current" {}
# random_id generates an 8-char hex suffix — Key Vault names must be globally
unique
resource "random_id" "kv_suffix" { byte_length=4 }

resource "azurerm_key_vault" "lab_kv" {
  name                       = "kv-fslab-${random_id.kv_suffix.hex}"
  location                   = azurerm_resource_group.rg.location
  resource_group_name        = azurerm_resource_group.rg.name
  tenant_id                  = data.azurerm_client_config.current.tenant_id
  sku_name                   = "standard"
  enable_rbac_authorization = true   # REQUIRED — without this, 403 on all secret
ops
  soft_delete_retention_days = 7
  purge_protection_enabled   = false
  tags = { Environment="Lab" ManagedBy="Terraform" }
}

# Grant whoever ran az login permission to write secrets.
# current.object_id resolves dynamically to the logged-in identity.
resource "azurerm_role_assignment" "kv_deployer_access" {
  scope                = azurerm_key_vault.lab_kv.id
  role_definition_name = "Key Vault Secrets Officer"
  principal_id         = data.azurerm_client_config.current.object_id
}

# depends_on ensures the role assignment propagates before Terraform
# tries to write the secret — without this you get a 403 error.
resource "azurerm_key_vault_secret" "admin_password" {
  name         = "vm-admin-password"
  value        = var.admin_password
  key_vault_id = azurerm_key_vault.lab_kv.id
  depends_on   = [azurerm_role_assignment.kv_deployer_access]
  tags         = { ManagedBy="Terraform" }
}
```

outputs.tf

Printed to your terminal after terraform apply. Copy key_vault_name immediately — you pass it to configure-lab.ps1 in Step 8.

```
output "dc01_public_ip"     { value=azurerm_public_ip.dc01.ip_address
description="DC01 public IP." }
output "dc01_private_ip"    {
value=azurerm_network_interface.dc01.private_ip_address description="Always
10.0.1.4 — DC01 static DNS address." }
output "fs01_public_ip"     { value=azurerm_public_ip.fs01.ip_address
description="FS01 public IP." }
output "client01_public_ip" { value=azurerm_public_ip.client01.ip_address
description="RDP here as test users to verify the lab." }
output "key_vault_name"     { value=azurerm_key_vault.lab_kv.name
description="Pass to configure-lab.ps1 with -KeyVaultName." }
```

terraform.tfvars.example

```
location       = "Central US"
rdp_source     = "YOUR_PUBLIC_IP/32"   # Find at whatismyip.com — format:
1.2.3.4/32
server_vm_size = "Standard_B2as_v2"
client_vm_size = "Standard_B2as_v2"

# admin_password is NOT set here — use an environment variable:
#   PowerShell: $env:TF_VAR_admin_password = "YourStrongPassword!"
#   Bash:       export TF_VAR_admin_password="YourStrongPassword!"
# Min 12 chars, upper + lower + number + symbol.
```

.gitignore

```
terraform.tfvars
*.tfvars
!terraform.tfvars.example
terraform.tfstate
terraform.tfstate.backup
*.tfstate
.terraform/
.terraform.lock.hcl
*.tfplan
.DS_Store
Thumbs.db
```

Step 3 — One-Time Remote State Setup

Run these commands once. If you already have a RG-TerraformState storage account from a previous lab, just update backend.tf with the existing name.

```
az group create --name RG-TerraformState --location "Central US"

# Name must be globally unique, 3-24 chars, lowercase letters and numbers only
az storage account create \
  --name tfstatentfslab \
  --resource-group RG-TerraformState \
  --sku Standard_LRS \
  --encryption-services blob

az storage container create --name tfstate --account-name tfstatentfslab

# Verify — must show: tfstate
az storage container list --account-name tfstatentfslab --query "[].name" -o tsv
```

Update backend.tf now. Open backend.tf and replace REPLACE_WITH_YOUR_STORAGE_ACCOUNT_NAME with the name above. Do this before running terraform init.

Step 4 — Configure Variables

```
Copy-Item terraform.tfvars.example terraform.tfvars
# Open terraform.tfvars and set rdp_source to your IP from whatismyip.com

# Set admin password as environment variable — never put it in a file
$env:TF_VAR_admin_password = "YourStrongPassword!"
echo $env:TF_VAR_admin_password   # If nothing prints, set it again before apply
```

Step 5 — Deploy Infrastructure

```
az login
terraform init    # Downloads providers, connects to remote state
terraform plan    # Review — expect 12-15 resources including 3 VMs
terraform apply   # Type yes — takes 10-15 minutes

# After apply — copy the Key Vault name for Step 8
terraform output key_vault_name
```

Output	Write it here
key_vault_name — needed for Step 8	
client01_public_ip — for RDP in Step 9	

Step 6 — Create the Scripts

Create each file inside your scripts/ folder. These are pushed to VMs automatically by configure-lab.ps1 — you never run them by hand.

scripts/00-promote-dc.ps1

Installs AD DS and promotes DC01 to a Domain Controller for lab.local. SAFE_MODE_PASSWORD is replaced at runtime by configure-lab.ps1 with the password from Key Vault. DC01 reboots automatically after promotion — configure-lab.ps1 detects the disconnect and waits for it to come back before proceeding to Step 2.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
```

```
Write-Host "Installing AD DS..." -ForegroundColor Yellow
Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools
-Verbose:$false
$safeModePassword = ConvertTo-SecureString "SAFE_MODE_PASSWORD" -AsPlainText -Force
Import-Module ADDSDeployment
Install-ADDSForest `
    -DomainName                    "lab.local" `
    -DomainNetbiosName             "LAB" `
    -ForestMode                    "WinThreshold" `
    -DomainMode                    "WinThreshold" `
    -InstallDns:                   $true `
    -SafeModeAdministratorPassword $safeModePassword `
    -Force:                        $true `
    -NoRebootOnCompletion:         $false
# DC01 reboots here. configure-lab.ps1 waits for it to come back online.
```

scripts/01-create-ad-users-groups.ps1

Creates three OUs, four security groups, and five test users. Each user is assigned to their group. Group membership drives NTFS permissions — add a user to GRP_Finance and they automatically inherit all Finance share access without any additional permission changes.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
$domain="lab.local"; $domainDN="DC=lab,DC=local"
$password=ConvertTo-SecureString "P@ssw0rd123!" -AsPlainText -Force
foreach ($ou in @("Lab Users","Lab Computers","Lab Groups")) {
    New-ADOrganizationalUnit -Name $ou -Path $domainDN
-ProtectedFromAccidentalDeletion $false
    Write-Host "Created OU: $ou" -ForegroundColor Yellow
}
foreach ($group in @("GRP_Finance","GRP_HR","GRP_Sales","GRP_IT")) {
    New-ADGroup -Name $group -GroupScope Global -GroupCategory Security -Path
"OU=Lab Groups,$domainDN"
    Write-Host "Created group: $group" -ForegroundColor Yellow
}
$users=@(
    @{First="John"; Last="Smith"; Username="john.smith";  Group="GRP_IT"     },
    @{First="Sarah";Last="Jones"; Username="sarah.jones"; Group="GRP_Finance" },
    @{First="Mike"; Last="Brown"; Username="mike.brown";  Group="GRP_Finance" },
    @{First="Lisa"; Last="White"; Username="lisa.white";  Group="GRP_HR"      },
    @{First="Tom";  Last="Davis"; Username="tom.davis";   Group="GRP_Sales"   })
foreach ($user in $users) {
    New-ADUser -GivenName $user.First -Surname $user.Last -Name "$($user.First)
$($user.Last)" `
               -SamAccountName $user.Username -UserPrincipalName
"$($user.Username)@$domain" `
               -Path "OU=Lab Users,$domainDN" -AccountPassword $password -Enabled
$true -PasswordNeverExpires $true
    Add-ADGroupMember -Identity $user.Group -Members $user.Username
    Write-Host "Created: $($user.Username) -> $($user.Group)" -ForegroundColor Cyan
}
Write-Host "`nDone." -ForegroundColor Green
```

scripts/02-configure-shares-and-permissions.ps1

Creates four folders, shares them as SMB shares, then applies NTFS permissions per group. Share-level permission is **Everyone Full Control** — intentional. Real security comes from

NTFS, not the share. **(OI)(CI)** = Object Inherit + Container Inherit — files and subfolders inside each share automatically inherit these permissions.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
$domain="LAB"; $basePath="C:\Shares"
New-Item -Path $basePath -ItemType Directory -Force
foreach ($folder in @("Finance","HR","Sales","IT")) {
    New-Item -Path "$basePath\$folder" -ItemType Directory -Force
    New-SmbShare -Name $folder -Path "$basePath\$folder" -FullAccess "Everyone"
}
function Set-FolderPermissions { param($path,$permissions)
    icacls $path /inheritance:d
    icacls $path /remove "BUILTIN\Users"
    icacls $path /remove "Everyone"
    icacls $path /remove "NT AUTHORITY\Authenticated Users"
    foreach ($p in $permissions) { icacls $path /grant
"$($p.Identity)`:$($p.Rights)" }
}
Set-FolderPermissions -path "$basePath\Finance" -permissions @(
    @{Identity="$domain\GRP_Finance";Rights="(OI)(CI)M"},
    @{Identity="$domain\GRP_HR";Rights="(OI)(CI)R"},
    @{Identity="$domain\GRP_IT";Rights="(OI)(CI)F"},
    @{Identity="BUILTIN\Administrators";Rights="(OI)(CI)F"})
Set-FolderPermissions -path "$basePath\HR" -permissions @(
    @{Identity="$domain\GRP_HR";Rights="(OI)(CI)M"},
    @{Identity="$domain\GRP_IT";Rights="(OI)(CI)F"},
    @{Identity="BUILTIN\Administrators";Rights="(OI)(CI)F"})
Set-FolderPermissions -path "$basePath\Sales" -permissions @(
    @{Identity="$domain\GRP_Sales";Rights="(OI)(CI)M"},
    @{Identity="$domain\GRP_IT";Rights="(OI)(CI)F"},
    @{Identity="BUILTIN\Administrators";Rights="(OI)(CI)F"})
Set-FolderPermissions -path "$basePath\IT" -permissions @(
    @{Identity="$domain\GRP_IT";Rights="(OI)(CI)F"},
    @{Identity="BUILTIN\Administrators";Rights="(OI)(CI)F"})
Write-Host "`nDone." -ForegroundColor Cyan
```



scripts/03-configure-rdp-gpo.ps1

Creates a GPO that enables RDP on machines in the Lab Computers OU and moves CLIENT01's computer account into that OU. Adding domain users to the Remote Desktop Users group is handled separately by 06-add-rdp-users.ps1 because WinRM is blocked by the NSG — Invoke-Command from DC01 to CLIENT01 would fail.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
$gpoName="Lab - Allow RDP for Domain Users"; $ouPath="OU=Lab
Computers,DC=lab,DC=local"
New-GPO -Name $gpoName | Out-Null
New-GPLink -Name $gpoName -Target $ouPath
Set-GPRegistryValue -Name $gpoName -Key
"HKLM\System\CurrentControlSet\Control\Terminal Server" -ValueName
"fDenyTSConnections" -Type DWord -Value 0
$computer=Get-ADComputer -Filter { Name -eq "CLIENT01" } -ErrorAction
SilentlyContinue
if ($computer) { $computer | Move-ADObject -TargetPath $ouPath; Write-Host "Moved
CLIENT01 to Lab Computers OU." -ForegroundColor Cyan }
else { Write-Warning "CLIENT01 not yet in AD — may still be joining." }
Write-Host "`nGPO configuration complete." -ForegroundColor Green
```

scripts/04-domain-join.ps1

Used for both FS01 and CLIENT01. Sets DNS to DC01's static IP (10.0.1.4) first so lab.local can resolve. Retries DNS resolution every 15 seconds for up to 3 minutes — this handles the timing gap between DC01 finishing promotion and FS01/CLIENT01 attempting to join. ADMIN_PASSWORD is replaced at runtime by configure-lab.ps1.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
$adapter=Get-NetAdapter | Where-Object{$_.Status -eq "Up"} | Select-Object -First 1
Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses
"10.0.1.4"
$retries=0; $resolved=$false
do { Start-Sleep -Seconds 15; $retries++
    $resolved=[bool](Resolve-DnsName "lab.local" -ErrorAction SilentlyContinue)
    Write-Host "  Attempt $retries -- resolved: $resolved"
} while (-not $resolved -and $retries -lt 12)
if (-not $resolved) { throw "lab.local did not resolve after 3 minutes." }
$domainCred=New-Object PSCredential("LAB\azureadmin",
    (ConvertTo-SecureString "ADMIN_PASSWORD" -AsPlainText -Force))
Add-Computer -DomainName "lab.local" -Credential $domainCred -Restart -Force
# VM restarts here. configure-lab.ps1 waits for it to come back.
```

scripts/05-verify-ad.ps1

Automated verification of all OUs, groups, and user memberships. Prints [PASS] or [FAIL] for each check. Runs as the final stage of configure-lab.ps1 so you get a complete status report without logging into DC01.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
Import-Module ActiveDirectory
$pass=$true
Write-Host "`n=== Active Directory Verification ===" -ForegroundColor Cyan
foreach ($ou in @("Lab Users","Lab Groups","Lab Computers")) {
    $exists=[bool](Get-ADOrganizationalUnit -Filter "Name -eq '$ou'" -ErrorAction
SilentlyContinue)
    if ($exists) { Write-Host "  [PASS] OU: $ou" -ForegroundColor Green }
    else { Write-Host "  [FAIL] OU missing: $ou" -ForegroundColor Red; $pass=$false
}
}
foreach ($g in @("GRP_Finance","GRP_HR","GRP_Sales","GRP_IT")) {
    $exists=[bool](Get-ADGroup -Filter "Name -eq '$g'" -ErrorAction
SilentlyContinue)
    if ($exists) { Write-Host "  [PASS] Group: $g" -ForegroundColor Green }
    else { Write-Host "  [FAIL] Group missing: $g" -ForegroundColor Red;
$pass=$false }
}
$expectedUsers=@(

@{Username="john.smith";Group="GRP_IT"},@{Username="sarah.jones";Group="GRP_Finance
"},

@{Username="mike.brown";Group="GRP_Finance"},@{Username="lisa.white";Group="GRP_HR"
},
    @{Username="tom.davis";Group="GRP_Sales"})
foreach ($u in $expectedUsers) {
```

```
    $user=Get-ADUser -Filter "SamAccountName -eq '$($u.Username)'" -ErrorAction
SilentlyContinue
    if (-not $user) { Write-Host "  [FAIL] User missing: $($u.Username)"
-ForegroundColor Red; $pass=$false; continue }
    $members=Get-ADGroupMember -Identity $u.Group | Select-Object -ExpandProperty
SamAccountName
    if ($members -contains $u.Username) { Write-Host "  [PASS] $($u.Username) ->
$($u.Group)" -ForegroundColor Green }
    else { Write-Host "  [FAIL] $($u.Username) not in $($u.Group)" -ForegroundColor
Red; $pass=$false }
}
Write-Host "`n=== AD Verification $(if ($pass){"PASSED"}else{"FAILED"}) ==="
-ForegroundColor $(if ($pass){"Green"}else{"Red"})
```

scripts/05-verify-shares.ps1

Verifies all four SMB shares exist and every NTFS permission entry is correct. Uses bitwise AND for permission checks — Windows sometimes combines multiple rights into a single flags value so exact string matching would produce false failures.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
$basePath="C:\Shares"; $pass=$true
Write-Host "`n=== Share and NTFS Verification ===" -ForegroundColor Cyan
$expectedACLs=@{

"Finance"=@(@{Identity="LAB\GRP_Finance";Right=[System.Security.AccessControl.FileS
ystemRights]::Modify}

@{Identity="LAB\GRP_HR";Right=[System.Security.AccessControl.FileSystemRights]::Rea
d}

@{Identity="LAB\GRP_IT";Right=[System.Security.AccessControl.FileSystemRights]::Ful
lControl})

"HR"=@(@{Identity="LAB\GRP_HR";Right=[System.Security.AccessControl.FileSystemRight
s]::Modify}

@{Identity="LAB\GRP_IT";Right=[System.Security.AccessControl.FileSystemRights]::Ful
lControl})

"Sales"=@(@{Identity="LAB\GRP_Sales";Right=[System.Security.AccessControl.FileSyste
mRights]::Modify}

@{Identity="LAB\GRP_IT";Right=[System.Security.AccessControl.FileSystemRights]::Ful
lControl})

"IT"=@(@{Identity="LAB\GRP_IT";Right=[System.Security.AccessControl.FileSystemRight
s]::FullControl})}
foreach ($share in $expectedACLs.Keys) {
    Write-Host "`n[ $share ]" -ForegroundColor White
    $smb=Get-SmbShare -Name $share -ErrorAction SilentlyContinue
    if ($smb) { Write-Host "  [PASS] SMB share exists" -ForegroundColor Green }
    else { Write-Host "  [FAIL] Share missing" -ForegroundColor Red; $pass=$false;
continue }
    $acl=(Get-Acl "$basePath\$share").Access
    foreach ($expected in $expectedACLs[$share]) {
        $ace=$acl|Where-Object{$_.IdentityReference.Value -eq $expected.Identity
-and $_.AccessControlType -eq "Allow"}
        if (-not $ace) { Write-Host "  [FAIL] $($expected.Identity) has no entry"
-ForegroundColor Red; $pass=$false; continue }
        $hasRight=($ace.FileSystemRights -band $expected.Right) -eq $expected.Right
```

```
        if ($hasRight) { Write-Host "  [PASS] $($expected.Identity) ->
$($expected.Right)" -ForegroundColor Green }
        else { Write-Host "  [FAIL] $($expected.Identity) wrong rights"
-ForegroundColor Red; $pass=$false }
    }
}
Write-Host "`n=== Verification $(if ($pass){"PASSED"}else{"FAILED"}) ==="
-ForegroundColor $(if ($pass){"Green"}else{"Red"})
```

scripts/06-add-rdp-users.ps1

Adds LAB\Domain Users to the local Remote Desktop Users group on CLIENT01. Runs directly on CLIENT01 via az vm run-command because WinRM (port 5985) is blocked by the NSG. Safe to re-run — checks if the member already exists before adding.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
$rdpGroup="Remote Desktop Users"; $domainUsers="LAB\Domain Users"
$existing=Get-LocalGroupMember -Group $rdpGroup -ErrorAction SilentlyContinue |
Where-Object{$_.Name -eq $domainUsers}
if ($existing) { Write-Host "$domainUsers already in $rdpGroup." -ForegroundColor
Yellow }
else { Add-LocalGroupMember -Group $rdpGroup -Member $domainUsers
    Write-Host "Added $domainUsers to $rdpGroup." -ForegroundColor Green }
Get-LocalGroupMember -Group $rdpGroup | Select-Object Name,ObjectClass |
Format-Table -AutoSize
```

Step 7 — Create the Orchestration Script

Create this file in the project root — not in scripts/. It coordinates all seven stages by pushing each script to the right VM through the Azure agent using az vm run-command.

configure-lab.ps1

```
param([Parameter(Mandatory=$true)][string]$KeyVaultName,[string]$ResourceGroup="RG-
FileServerLab")
$startTime=Get-Date
Write-Host "`n[$(Get-Date -Format "HH:mm:ss")] Retrieving credentials from Key
Vault..." -ForegroundColor Cyan
$AdminPassword=az keyvault secret show --vault-name $KeyVaultName --name
"vm-admin-password" --query "value" -o tsv
if (-not $AdminPassword -or $LASTEXITCODE -ne 0) { throw "Could not retrieve
password from Key Vault. Run az login first." }
Write-Host "  Credentials retrieved." -ForegroundColor Green

function Invoke-VMScript {
param([string]$VMName,[string]$ScriptPath,[string]$Description,[hashtable]$Replacem
ents=@{})
    Write-Host "`n[$(Get-Date -Format "HH:mm:ss")] >>> $Description"
-ForegroundColor Cyan
```

```
    $script=Get-Content $ScriptPath -Raw
    foreach ($key in $Replacements.Keys) { $script=$script -replace
$key,[regex]::Escape($Replacements[$key]) }

$tempFile=[System.IO.Path]::GetTempPath()+[System.IO.Path]::GetRandomFileName()+".p
s1"
    $script | Out-File -FilePath $tempFile -Encoding UTF8
    try {
        $jsonLines=az vm run-command invoke --resource-group $ResourceGroup --name
$VMName --command-id RunPowerShellScript --scripts "@$tempFile" --output json
--only-show-errors
        if ($LASTEXITCODE -ne 0) { throw "az vm run-command failed on $VMName" }
        $result=($jsonLines -join "`n") | ConvertFrom-Json
        $stdout=($result.value | Where-Object{$_.code -like "*StdOut*"}).message
        $stderr=($result.value | Where-Object{$_.code -like "*StdErr*"}).message
        if ($stdout) { Write-Host $stdout }
        if ($stderr -and $stderr.Trim() -ne "") { Write-Warning "  VM StdErr:
$stderr" }
    } finally { Remove-Item $tempFile -ErrorAction SilentlyContinue }
}

function Wait-VMOnline { param([string]$VMName,[int]$TimeoutSeconds=360)
    Write-Host "  Waiting for $VMName..." -ForegroundColor Yellow
    $elapsed=0
    do { Start-Sleep -Seconds 15; $elapsed+=15
        try { $state=(az vm show -g $ResourceGroup -n $VMName -d --query
"powerState" -o tsv --only-show-errors 2>$null).Trim() } catch { $state="" }
        Write-Host "    $VMName -> $state ($elapsed s)" -ForegroundColor DarkGray
    } while ($state -ne "VM running" -and $elapsed -lt $TimeoutSeconds)
    if ($state -ne "VM running") { throw "Timeout: $VMName did not return within
${TimeoutSeconds}s" }
    Write-Host "  $VMName is online." -ForegroundColor Green
}

Write-Host "`n[STEP 1] Promoting DC01 to Domain Controller" -ForegroundColor
Magenta
try { Invoke-VMScript -VMName "DC01" -ScriptPath ".\scripts\00-promote-dc.ps1"
-Description "Promoting DC01" -Replacements @{"SAFE_MODE_PASSWORD"=$AdminPassword}
}
catch { Write-Host "  DC01 disconnected -- expected after promotion."
-ForegroundColor Yellow }
Start-Sleep -Seconds 60; Wait-VMOnline -VMName "DC01"; Start-Sleep -Seconds 90

Write-Host "`n[STEP 2] Creating OUs, Groups, and Users" -ForegroundColor Magenta
Invoke-VMScript -VMName "DC01" -ScriptPath
".\scripts\01-create-ad-users-groups.ps1" -Description "Creating AD objects"

Write-Host "`n[STEP 3] Joining FS01 to lab.local" -ForegroundColor Magenta
Invoke-VMScript -VMName "FS01" -ScriptPath ".\scripts\04-domain-join.ps1"
-Description "Joining FS01" -Replacements @{"ADMIN_PASSWORD"=$AdminPassword}
Start-Sleep -Seconds 30; Wait-VMOnline -VMName "FS01"; Start-Sleep -Seconds 30

Write-Host "`n[STEP 4] Configuring shares and NTFS on FS01" -ForegroundColor
Magenta
Invoke-VMScript -VMName "FS01" -ScriptPath
".\scripts\02-configure-shares-and-permissions.ps1" -Description "Creating shares
and NTFS"

Write-Host "`n[STEP 5] Joining CLIENT01 to lab.local" -ForegroundColor Magenta
Invoke-VMScript -VMName "CLIENT01" -ScriptPath ".\scripts\04-domain-join.ps1"
-Description "Joining CLIENT01" -Replacements @{"ADMIN_PASSWORD"=$AdminPassword}
Start-Sleep -Seconds 30; Wait-VMOnline -VMName "CLIENT01"; Start-Sleep -Seconds 30

Write-Host "`n[STEP 5b] Granting Domain Users RDP on CLIENT01" -ForegroundColor
Magenta
Invoke-VMScript -VMName "CLIENT01" -ScriptPath ".\scripts\06-add-rdp-users.ps1"
-Description "Adding Domain Users to RDP group"
```

```
Write-Host "`n[STEP 6] Configuring RDP GPO on DC01" -ForegroundColor Magenta
Invoke-VMScript -VMName "DC01" -ScriptPath ".\scripts\03-configure-rdp-gpo.ps1"
-Description "Creating RDP GPO"

Write-Host "`n[STEP 7] Running automated verification" -ForegroundColor Magenta
Invoke-VMScript -VMName "DC01" -ScriptPath ".\scripts\05-verify-ad.ps1"
-Description "Verifying AD"
Invoke-VMScript -VMName "FS01" -ScriptPath ".\scripts\05-verify-shares.ps1"
-Description "Verifying shares"

$duration=(Get-Date)-$startTime
Write-Host "`n=== LAB FULLY CONFIGURED ($([math]::Round($duration.TotalMinutes,1))
min) ===" -ForegroundColor Green
Write-Host "RDP into CLIENT01 as: LAB\sarah.jones  Password: P@ssw0rd123!"
```

Step 8 — Run the Lab Configuration

```
# Replace kv-fslab-XXXXXXXX with your key_vault_name from Step 5
.\configure-lab.ps1 -KeyVaultName "kv-fslab-XXXXXXXX"
# Takes 15-20 minutes, fully unattended
```

Step 9 — Verify the Lab

RDP into CLIENT01 using the public IP from terraform output. Test each scenario. All test user passwords are P@ssw0rd123!

Log in as	Share	Expected	Why
LAB\sarah.jones	\\FS01\Finance	✅ Read and write	Member of GRP_Finance — Modify NTFS
LAB\sarah.jones	\\FS01\HR	❌ Access Denied	Not in GRP_HR — no ACE on HR share
LAB\lisa.white	\\FS01\Finance	✅ Read only	GRP_HR has Read on Finance share
LAB\lisa.white	\\FS01\HR	✅ Read and write	Member of GRP_HR — Modify NTFS
LAB\john.smith	\\FS01\IT	✅ Full Control	Member of GRP_IT — Full Control NTFS
LAB\tom.davis	\\FS01\Finance	❌ Access Denied	GRP_Sales has no entry on Finance

Step 10 — Pause or Tear Down

Continuing to Lab 2? Stop the VMs — do not destroy. Lab 2 needs RG-FileServerLab and FS01 to exist. Stopped VMs have no compute charges.

```
# Pause — no compute charges while stopped
az vm stop --ids $(az vm list -g RG-FileServerLab --query "[].id" -o tsv) --no-wait
# Restart before Lab 2:
az vm start --ids $(az vm list -g RG-FileServerLab --query "[].id" -o tsv)
--no-wait

# Full teardown — only when completely done with both labs
terraform destroy
```

Troubleshooting

Problem	Cause	Solution
configure-lab.ps1 fails at Key Vault	Session expired or missing role	Run az login, retry. Confirm Key Vault Secrets User role on the vault
Domain join fails — DNS not resolving	DC01 still finishing promotion	Wait 2 minutes and re-run — configure-lab.ps1 resumes from where it stopped
Cannot RDP to VMs	IP changed or rdp_source mismatch	Run curl ifconfig.me, update terraform.tfvars rdp_source, run terraform apply
Access Denied unexpected	User not in the right group	Inside RDP run: whoami /groups to confirm group membership
GPO not applying	Policy cache not refreshed	Run: gpupdate /force inside the VM, then gpresult /r
terraform init fails	backend.tf not updated	Open backend.tf, confirm storage account name is correct and container exists